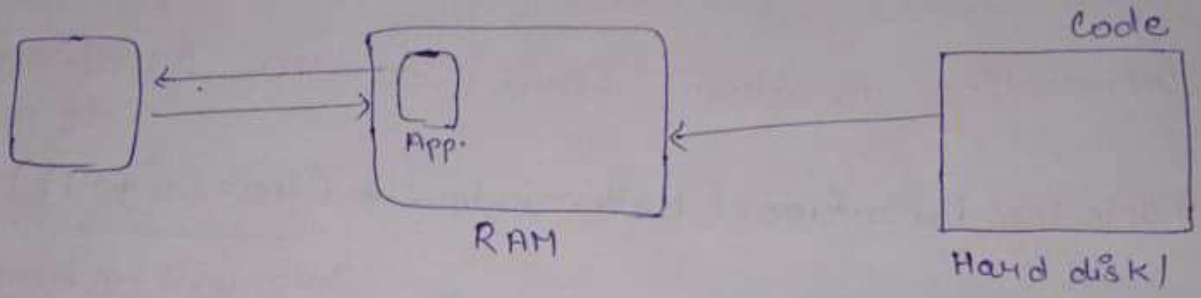
Javascript (lecture - 03)
Memory Management (Stack vs Heap)

* RAM, Harddisk and CPU Interaction

→ Every application is store in Hard disk / SSD.

→ At the end every application is a code.



→ To run the application it will be load in RAM

→ And CPU will run it.

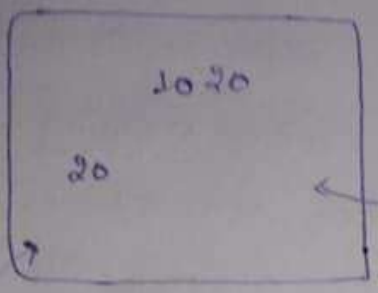
Why we load first in RAM ?

- If we directly load from SSD to CPU then it will take more time because SSD is slow.
- Loading the data from RAM to CPU is fast and easy process.

That's why RAM is costly to create.

Hard disk is permanent storage and RAM is temporary storage (Once you switch off the phone it will be gone)

Problem Statement: Managing Data Without unique identifiers.



a = 10 } when we run the code
b = 20 } they will get the memory in RAM

Store in RAM

RAM: 16 Byte (Assume)

a = 20 (You made some changes and made a=10 to a=20)

Now you want to change b = 20 to b = 25

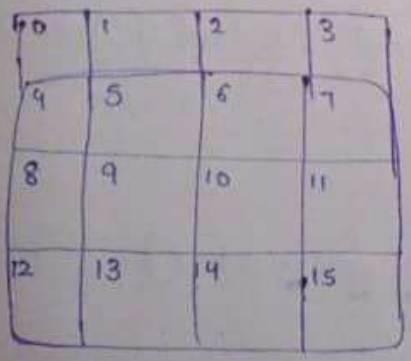
b = 25

Now Problem arises:

→ Which 20 will be change to 25.

→ What if you change the wrong 20. then data will be corrupted

Solution: Byte Addressable Memory



16 Byte RAM
(Byte Addressable)

Byte Addressable: Means every byte will get the address

1 Byte = 8 bit

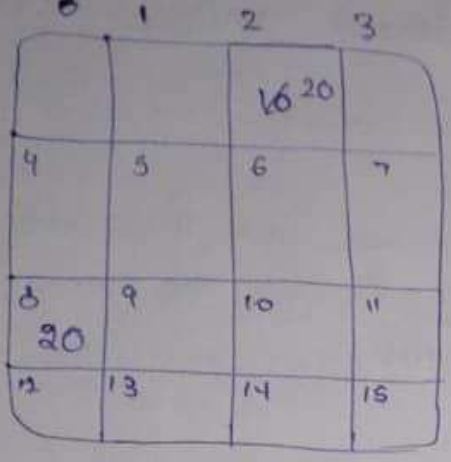
1 KB = 1024 B = 2^{10}

1 MB = 1024 KB = 2^{20}

1 GB = 1024 MB = 2^{30}

1 TB = 1024 GB = 2^{40}

Address - Simple giving number so we can uniquely identify



2 ← a = 10 ← 1 Byte (Assume it taking 1 Byte space in memory)

b = 20

8

Now if we save it in memory we have to do some mapping between the number and address. So it will easy to uniquely identify and change its value.

a = 20 ↓ 2 } we will go on address - 2 and change the 10 → 20

Next if you save the data in memory you will also see the address of variable corresponding to value

16 Byte = 16 Address
32 Byte = 32 Address
} n byte will have n addresses

→ How many bits will be require to save this address because at the end it will be also save in binary.

→ 4 bit = 2^4 (16 Address generated)

0000
0001
0010
0011
⋮
1111

For 32 byte how many bits are required?

→ $2^5 = 32$ (5 bits are required)

* How Program Use Addresses?
* Does we do the mapping between the address and variable?

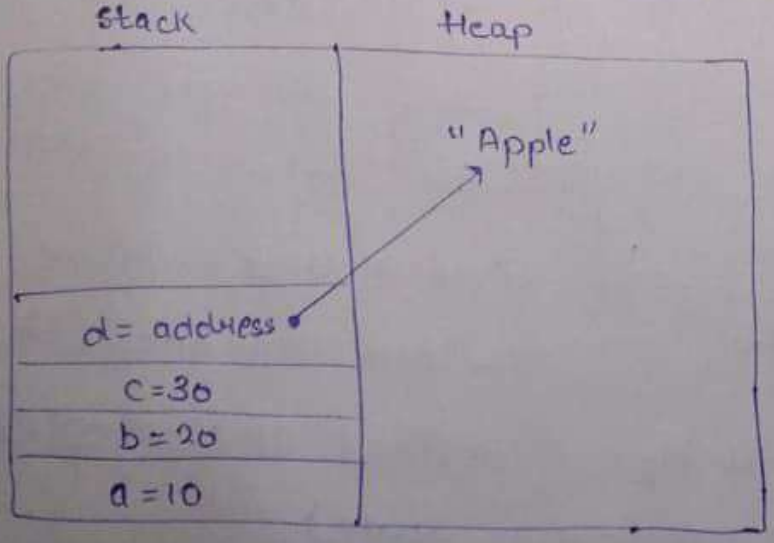
→ No, Computer do it own.

a = 10
↓
0010 = 10

This a, b, c are only shown to programmer. When the code run their addresses will be resolved means variable a will be replace by its address - 0010

There is no need of mapping table now. We can run the code in anyway. These variable name only shown to programmer.

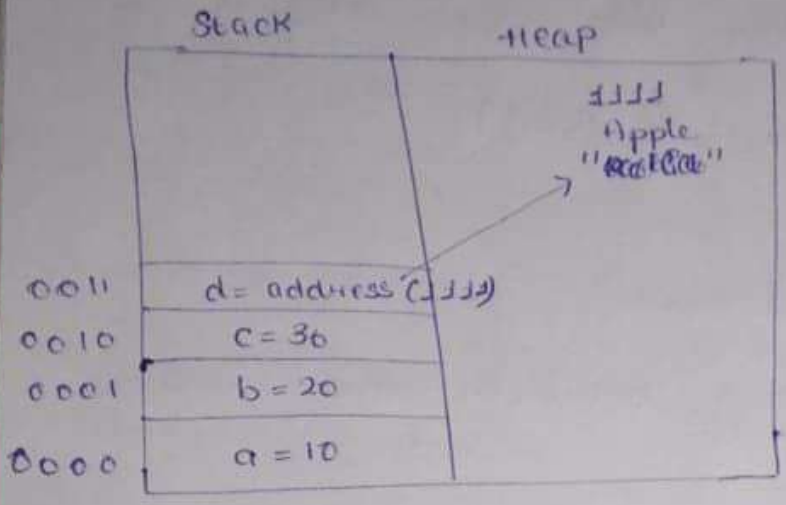
Introduction To Stack and Heap



Stack - Memory Allocation done by one by one.
Stack size - In KB

Heap - size will be in MB, GB

d = "Apple"
We can store it anywhere in heap but its address will be store in stack



In stack we can directly store the value as well as address

0000 ← a = 10

0001 ← b = 20

0010 ← C = 30

0011 ← d = "Apple"

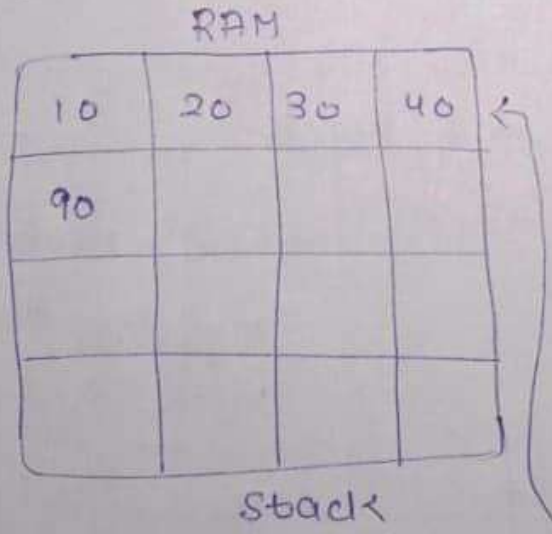
Accessing "Apple" — In 2 step

- first 0011 address
- then again address (1111)

Note

Whatever the address of Heap it will be store in stack.

Why Stack and Heap Exist?



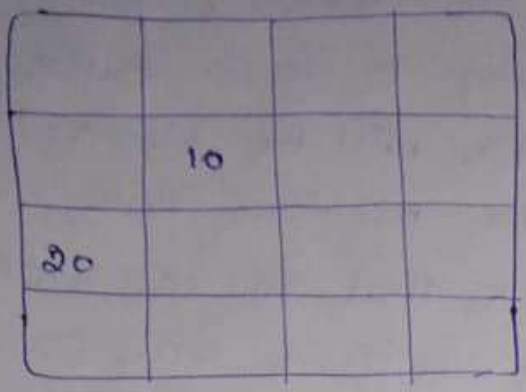
a = 10

b = 20

How will you store the data in RAM?

First Method - Store it one by one (You defined this simple rule)

Next element will be Store after the previous one.

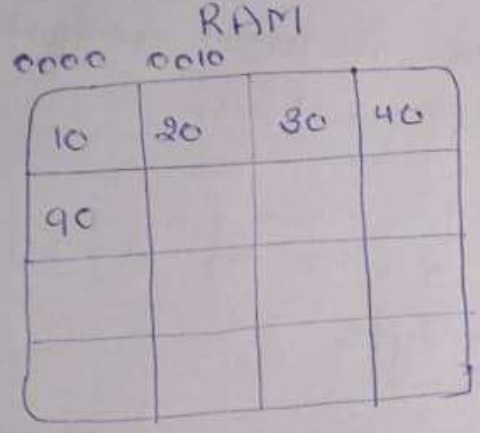


Heap

Second Method - store it randomly where the space is avilable

} Memory don't look like rows and columns. This is only the representation way

Why Use Both Stack and Heap?



Stack

0000 ← a = 10

b = 20 → Change 20 → SON

0010

b = "SON" ←

We assume 10 → 1 Byte

and SON has 3 character so it will take 3 Byte

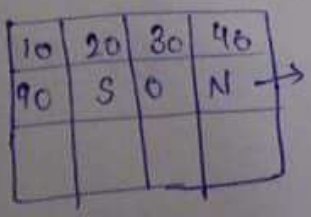
Now how will you insert the 3 Byte character into 1 Byte?

S → 01010011 X 20 X

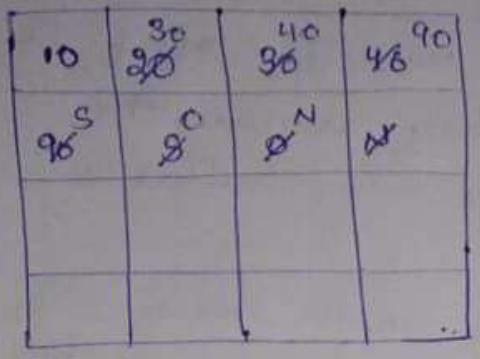
Occupied Occupied

Because if you save only S in position of 20 then where O, N will be save.

And also you defined the rule that next element will be store after the previous one.



You save the SON after 90. But still you can't delete 20 because if the space is empty you have the shift the data



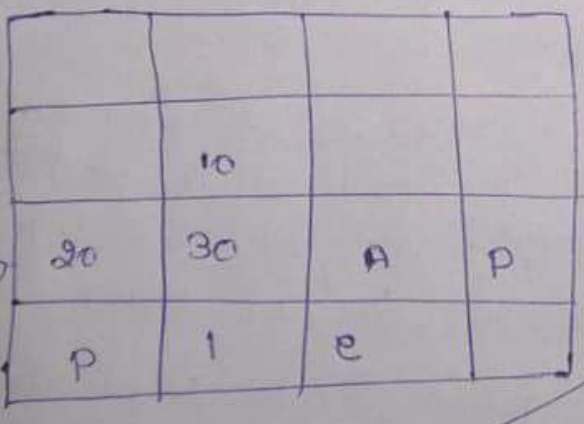
If 20 is deleted. But according to your rule the data will be store after the previous one. So you think that you can shift it but this is not the efficient way (Time Consuming)

Next problem - You have to change the addresses because you shift it one position behind.

→ Now if the same variable is used many times you have to changed it everywhere

→ You can easily change a = 10 to a = 20 because both will take 1 Byte

Same problem arise in Heap also.



0101 a = 10;

1000 b = 20;

1010 b = "Apple";

Now "Apple" can't save here because its next space is occupied

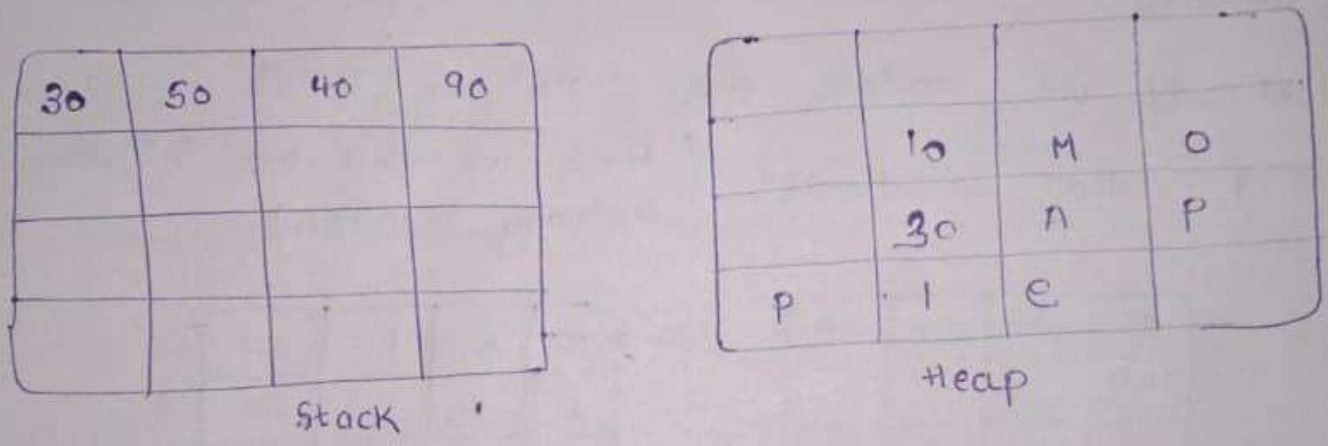
First we find the space for "Apple" then save it. And we can remove the b (1000) - 20 (this address) and allocate new memory (1010) to b.

But still there is a same problem. If b is used 10 times we have to update its address 10 times.

Fixed and Dynamic Data Allocation

We should save the fixed size data in stack because it will take the same memory.

Problem arise when the data is not fixed size like string (sometime 2, 8, 10, 20 ...)



f = "MO" → If we directly store the "MO" in Heap
0110 and save its address (0110)

After a ~~ave~~ while we updated it with "MO" → "Mohi"

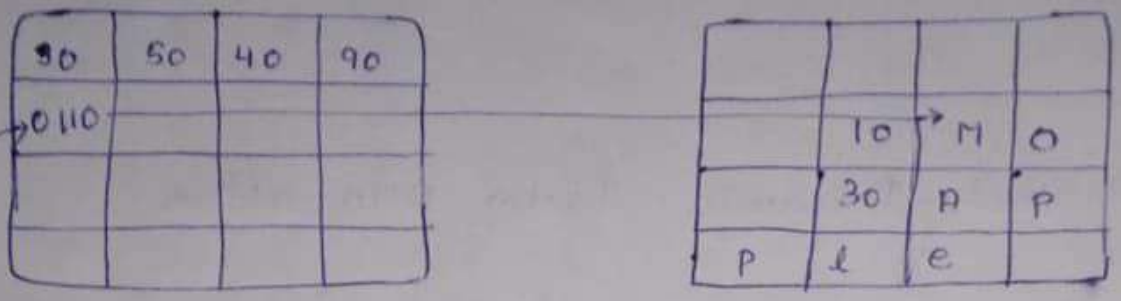
To save this 4 Byte memory location we have first find 4 Byte and then save "Mohi"

Due to this f address will be change and if it use 100 times then we have to updak it address by 100 times.

f = "Mohi"
0000

H	O	h	i
	10		
	30	A	P
P	1	e	

What if we store the address in stack?

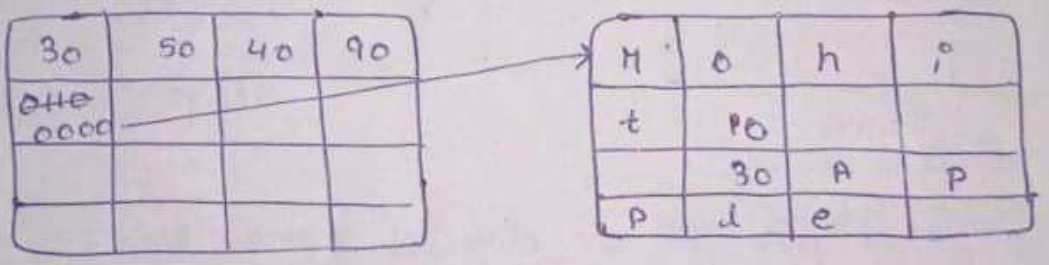


f = "Ho"
0100
and f is present at 4th location in memory

Here we save the address of this "Ho" in stack. As we know stack will store the fixed size ~~data type~~. So address size will be fixed

Next if we update the "Ho" → "Mohit"

f = "Ho" → "Mohit" (first we find the 5 Byte memory location)



f = "Mohit"
0100

And update the address in stack from 0110 → 0000

There is no changes in f (0100) location

If the f is written 100 times then there will be no change in memory location.

Note : This is a two step process if we access the "Mohit". first we go to this 0100 (stack) and from here we go to 0000. (Heap)

Why store addresses in stack?

→ If we directly store in address in heap and we update the & value from small Byte to large Byte then we have to update it address everywhere where the variable is used.

That's why we store the fixed size variable in stack and dynamic size in heap and its address in stack.

* Dynamic Data type in Javascript

→ String → Object → Array

Javascript Memory Allocation

You have listen that our system is 32 bit OS or 64 bit OS

32 bit OS - Address size will be fixed 4 Byte.
64 bit OS - Address size will be fixed of 8 Byte.

Like in 4 Bit

0000
0001
0010
:
1111

$2^4 = 16$ we can generate 16 addresses of different type

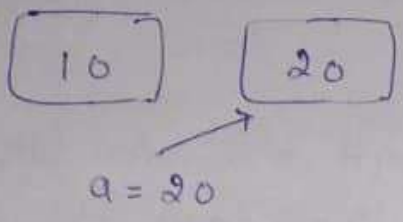
same for 32 bit

2^{32} - we can easily create 4 GB of RAM

4 Byte - 32 bit (Address size will be 32 bit)

Memory Allocation for Primitive Data type

Primitive data are immutable we cannot change it

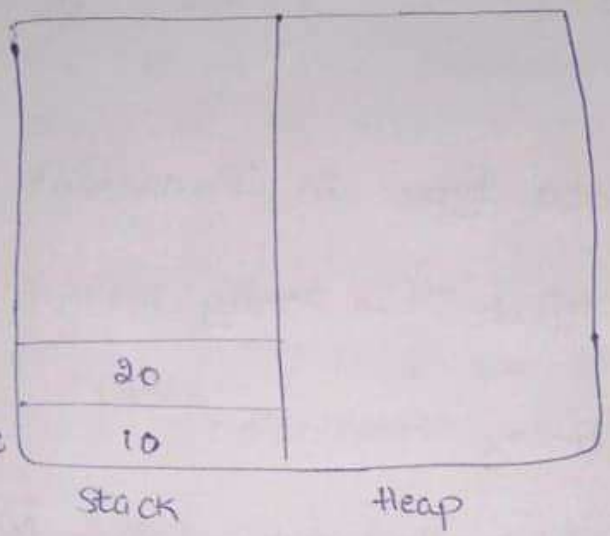


let a = 10

0xAbc

let b = 20

0xbcd = 20



We have variable a and b and we give the allocation to stack

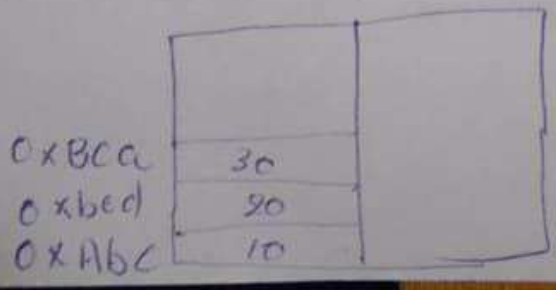
Update the a from 10 to 30

Variable name change to address and now you change the value 0xAbc = 10 → 30

go to this location and change to 30

But Rule say you cannot change 10 → 30 directly because they are immutable.

We have one choice left that we should create new memory allocation and write 30 there and the address (0xBca) which is generated replace by a.



0xAbc = 10

↓ Replace

0xBca = 10

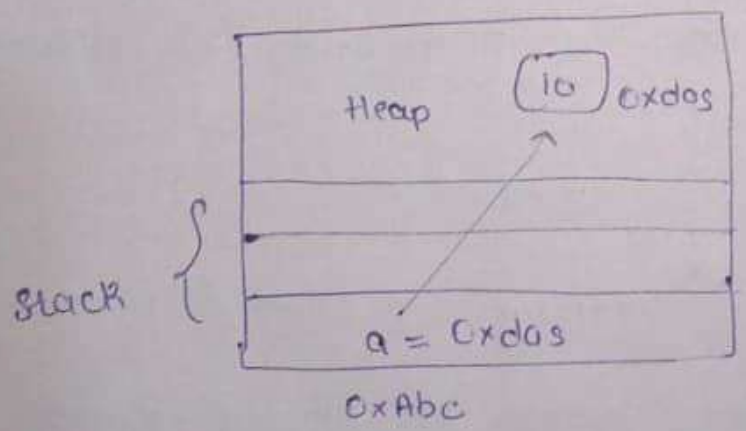
Problem arise - If variable (a) is used 10 time we have change it 10 times.

Because in primitive data type we cannot mutate or change it.

From this we understood that whether ~~store~~ it is fixed size or variable size, problem is same with both.

Although the size of data (10) is fixed that it will take 8 byte of memory but we cannot manipulate it.

That's why we store the primitive data type in Heap and store its address in stack



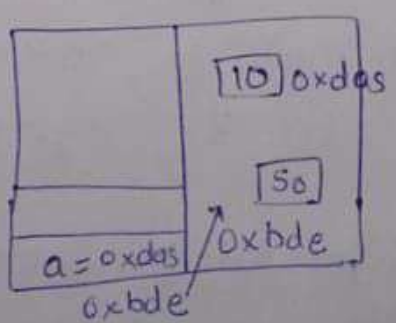
a = 10

0xAbc = 0xdas

This size will be 32 bit if we have 32 bit OS

Now we ~~change~~ can provide it different location for primitive data type in Heap

a = 10 50



a = 10

0xAbc = 0xdas

a = 50

0xAbc = 0xbde

Note: All Data will be saved in Heap whether they are primitive or non-primitive.

Optimizing Memory for true, false, null, and undefined.

When the program start the position of these four (true, false, null, undefined) will be fix. They will be already created.

Because if two variable use the "true" then we don't have to create it two times

Memory Allocation for objects

Same we will store it in Heap and its address in stack

Garbage Collection: Cleaning Up Unused memory

Javascript automatically delete the unused data if any pointer don't point it.

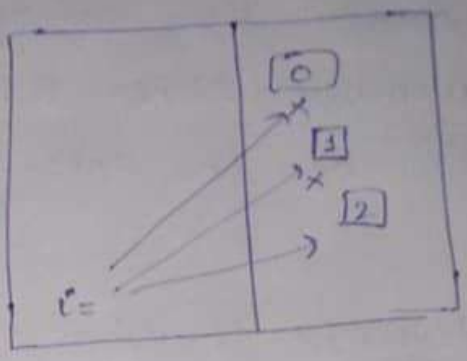
Garbage Collection first check in stack and understand which memory is in used and rest it will be deleted.

DeAllocation is important because program can be crash

Loop Optimization

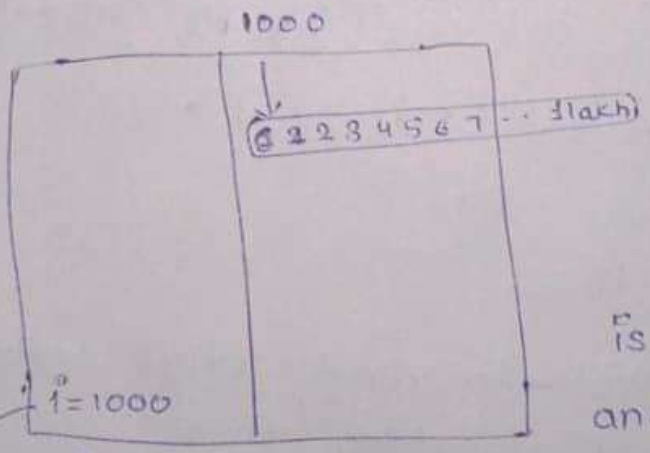
loop

```
for(i=0; i=100; i++)
{
    console.log(i);
}
```

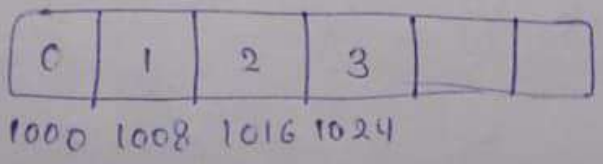


Now we have allocate the memory 100 times and memory allocation in heap is not easy because first we have to find the space to store the value and loop will be slow.

Strategy - 1 : We create the array of 1 lakh in advance and make its address fix in heap



Now if first element is at 1000 next will be at 1008 because this is byte addressable and in JS it take 8 Byte



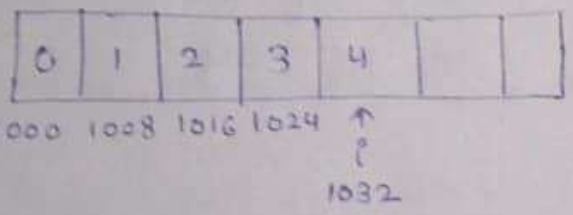
Element : Base Address + index * size of data

we can simply update the address here now because we create the array of size - 1 lakh

i = 1006
1008
1016

Problem with Strategy - 1 : • First we have to create the array of 1 Lakh and each number take 8 Byte memory so 1 Lakh Byte will be for this only

Alternate Solution : There is no need to create the array of 1 Lakh we can simply calculate its address by formula



As we know if i=3 and it become i=4 then its address will be 1032 we can calculate it

Element = Base Address + index * size

= 1000 + 4 * 8

= 1032

and if we want to read what value is present at that location then

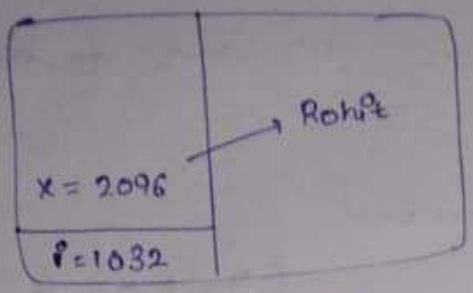
$$\frac{(1032 - 1000)}{8} = 4$$

We don't need to go to memory location in Heap. We can calculate it

$$\frac{(1040 - 1000)}{8} = 5$$

Because you know Base Address is fix and size of data is fix.

Encoding Data in Memory Addresses



(2096 - 1000)/8 = 30

We cannot apply this formula everywhere because if string is present then it will return number.

So what we can do that if 32-bit OS then

31 bit — 1 bit

0 (Number) Don't go to heap

1 (String) Go to heap location because the actual data will be present there

Problem Solved :

→ There is no need to create the array

→ We will read the last bit
- If it is zero then it will be indicate to number there is no need to go to heap
- If it is one then go to heap location

More Simplification : There is no need of base Address we can store the number in first 31-bit

2 - 0010 — — — — — — — 0 0 1 0 | 0

31 bit

It show which number is present at that location

There is no need of memory allocation. We manipulate it in address

Now we have 31 - bit so we can show 2^{31} numbers

If we have 3 - bit then we can generate 8 number

3 bit = 8 number

4 positive (0-3)

4 Negative (-1, -4)

(Half of them will be positive & Half of them will be negative)

2^{31} — $(2^{30} - 1)$ Positive (first bit will be 0)

$(-2^{30}$ ext) Negative (first bit will be 1)

But we have 8 Byte to represent a number and we left with 31 - bit

- So it will not store the all numbers like 32 bit, 33 bit, floating point numbers they will be store in Heap

This will be only for number which lie in this range

(-2^{30}) to $(2^{30} - 1)$

→ Small Integer number directly store in the form of address in stack

→ All will be stored in Heap